

Application Design Report

Project Overview

The application is a distributed **Campus Information Exchange System** developed in C++ using socket programming and multithreading. It follows a **client-server architecture** where multiple campus clients connect to a central server to exchange departmental messages and receive broadcast announcements.

The system uses:

- TCP communication for authentication and reliable message delivery
 - UDP communication for heartbeat monitoring and system-wide broadcasts
 - Multithreading to handle multiple clients and background services concurrently
-

System Architecture

The application consists of two major components:

1. Central Server
 2. Campus Client
-

1. Central Server Design

The server acts as the communication hub between all campuses.

Main Responsibilities

- Authenticate campuses
- Maintain connected client information
- Route messages between campuses
- Track online/offline status
- Send broadcast announcements
- Handle multiple clients simultaneously

Server Modules

A. Authentication Module

The server validates campus credentials using predefined campus names and passwords.

Functions Used

- `isValidCampus()`
- `authenticateCampus()`

Authentication Flow

Client sends:

LOGIN|Campus|Password

- 1.
2. Server validates:
 - Campus exists
 - Password matches
 - The campus is not already connected
3. Server responds:
 - `AUTH|SUCCESS`
 - or an authentication failure message

This prevents unauthorized access and duplicate logins.

B. TCP Communication Module

TCP is used for reliable communication between the server and clients.

Features

- Connection-oriented communication
- Reliable message delivery
- Ordered packet transmission

The server:

- Creates a listening socket on port `8080`

- Accepts multiple client connections
- Creates a separate thread for each client

Main Function

- `handleClient()`
-

C. Message Routing Module

This module forwards messages from one campus to another.

Message Format

MSG|TargetCampus|Department|Message

The server:

1. Validates packet structure
2. Verifies department and target campus
3. Finds destination socket
4. Forward a message using TCP

Forwarded Message Format

FROM|SenderCampus|Department|Message

D. UDP Heartbeat Module

UDP is used for lightweight periodic status updates.

Each client periodically sends:

HEARTBEAT|Campus

The server stores:

- Campus UDP address
- Last active timestamp
- Online status

Main Function

- `udpListener()`

This design allows the server to monitor active campuses efficiently without maintaining extra TCP overhead.

E. Broadcast Announcement Module

The administrator can send announcements to all online campuses.

Broadcast Format

BROADCAST|Announcement

The server sends the announcement using UDP to all registered campus addresses.

Main Function

- `broadcastAnnouncement()`
-

F. Admin Panel

The server contains a console-based admin panel.

Features

1. View campus statuses
2. Send broadcast announcements
3. Exit server

Main Function

- `adminPanel()`
-

G. Concurrency Design

The server uses multithreading for parallel operations.

Threads Used

Thread	Purpose
Main Thread	Accept client connections
Client Threads	Handle individual clients
UDP Thread	Receive heartbeat packets
Admin Thread	Run the admin console

H. Shared Resource Synchronization

Shared data structures:

- `connectedClients`
- `campusStatuses`

These are protected using:

`mutex clientMutex;`

The server uses `lock_guard<mutex>` to avoid race conditions during concurrent access.

2. Client Design

The client application represents a campus node.

Main Responsibilities

- Authenticate with the server
 - Send messages
 - Receive messages
 - Send heartbeat packets
 - Receive broadcasts
-

Client Modules

A. Authentication Module

The client:

1. Takes campus name and password input
2. Creates TCP connection
3. Sends login request

Main Function

- `authenticateClient()`
-

B. Message Sending Module

Users can send messages to other campuses.

Validation Includes

- Valid target campus
- Valid department
- Non-empty message
- Message length check

Main Function

- `sendMessage()`
-

C. Server Listener Module

A dedicated thread continuously listens for incoming TCP messages.

Handles

- Incoming campus messages
- Server errors
- Broadcast messages

Main Function

- `listenToServer()`
-

D. Heartbeat Module

A separate thread periodically sends UDP heartbeat packets every 10 seconds.

Main Function

- `sendHeartbeat()`

This helps the server determine whether the client is online.

E. Menu System

The client provides a console-based menu.

Features

1. Send Message
2. Exit

Main Function

- `showMenu()`
-

Communication Protocol Design

The application uses a simple text-based protocol.

Packet Formats

Packet Type	Format
Login	<code>`LOGIN</code>
Message	<code>`MSG</code>

Forwarded Message	`FROM
Heartbeat	`HEARTBEAT
Broadcast	`BROADCAST

Using delimiter-based packets makes parsing simple and lightweight.

Networking Design

TCP Usage

Used for:

- Authentication
- Reliable messaging

Reason

TCP guarantees ordered and reliable delivery.

UDP Usage

Used for:

- Heartbeats
- Broadcast announcements

Reason

UDP is faster and suitable for lightweight periodic packets where occasional loss is acceptable.

Data Structures Used

Server Side

Connected Clients

map<string, ClientInfo>

Stores:

- Campus name
 - Socket
 - Authentication status
-

Campus Statuses

map<string, CampusStatus>

Stores:

- Online status
 - UDP address
 - Last heartbeat time
-

Error Handling

The system includes validation and error handling for:

- Invalid campuses
- Invalid departments
- Empty messages
- Connection failures
- Packet format errors
- Offline target campuses

This improves reliability and prevents invalid operations.

Strengths of the Design

Advantages

- Clear separation of TCP and UDP responsibilities
 - Multithreaded architecture
 - Lightweight protocol design
 - Real-time communication
 - Campus status monitoring
 - Concurrent client handling
-

Conclusion

The application demonstrates a well-structured implementation of a distributed client-server communication system using socket programming in C++.

The design effectively combines:

- TCP reliability
- UDP efficiency
- Multithreading
- Synchronization mechanisms

to provide a functional real-time campus messaging platform.

The project is a strong example of practical networking concepts, including:

- Socket programming
- Concurrent systems
- Client-server architecture
- Communication protocols
- Network status monitoring